Girls to Code 2026: Comprehensive Student Guide & Syllabus

Programme Context & Pedagogical Philosophy

The Girls to Code 2026 Programme, launched in collaboration with Dhiraagu and Women in Tech Maldives, is an intensive, seven-week technical development journey tailored for young women aged 16–30 with zero prior coding experience. The initiative is designed to bridge the digital gender divide, fostering technical capability, innovation, and leadership in the Maldivian digital economy.

The Maldives faces unique geographic and urban challenges, ranging from the hyper-density of Malé—which houses over 94,000 people per square kilometer—to the complex logistics of delivering essential digital services across dispersed island communities. Addressing these challenges requires resilient, intelligence-driven digital infrastructure and a highly capable technical workforce. The curriculum positions technology as a tool for civic and community problem-solving, equipping participants to build solutions that improve mobility, resilience, and digital literacy across the nation.

Overcoming the Imposter Phenomenon

A core component of the instructional design addresses the Imposter Phenomenon—a persistent sense of intellectual inadequacy commonly experienced by high-achieving women entering highly technical fields. The pedagogy actively counteracts these feelings through:

- **Simulation-Based and Project-Based Learning:** Moving away from rote memorization toward collaborative, hands-on application to build authentic self-efficacy.
- **Peer-Interaction and Collaboration:** Normalizing errors and utilizing structured peer code reviews to demonstrate that debugging is a universal engineering reality, not a personal failure.
- **Progressive Wins:** Structuring the learning pathway to deliver small, visible technical victories early in the program, such as deploying a live webpage in the first week.

Course Overview & Expected Outcomes

The syllabus is strictly structured around a progressive learning pathway: **Web → Python → Data → Backend → AI → GitHub Collaboration → Deployment → Demo Day**. Participants will learn to merge design, code, and artificial intelligence into a functional product.

By the conclusion of the seven-week program, participants will successfully demonstrate the ability to achieve the following ten core outcomes:

1. **Frontend Engineering:** Build and style responsive, accessible webpages utilizing HTML5, CSS3, and Flexbox layouts.
2. **Web Interactivity:** Implement dynamic interactivity in web applications utilizing Vanilla JavaScript and the Document Object Model (DOM).
3. **Algorithmic Logic:** Write beginner-level Python programs, utilizing variables, loops, conditionals, and common data structures (lists, dictionaries).
4. **Data Science Fundamentals:** Clean, analyze, and visualize real-world datasets utilizing industry-standard Python libraries, including pandas, matplotlib, and seaborn.

5. **Backend Architecture:** Develop a dynamic web server utilizing the Python Flask micro-framework, successfully connecting frontend HTML forms to backend Python logic.
6. **Professional Version Control:** Master Git and GitHub workflows for local version control, collaborative team engineering, branching, and pull request management.
7. **AI-Assisted Engineering:** Integrate generative AI thoughtfully and responsibly, utilizing it for syntax debugging, UI generation, and programmatic task automation.
8. **Responsible Technology:** Navigate AI ethics and digital accessibility, implementing WCAG (Web Content Accessibility Guidelines) standards and safety guardrails against algorithmic bias and data exposure.
9. **Cloud Deployment:** Deploy a functional, full-stack Minimum Viable Product (MVP) to the live internet utilizing the Render cloud hosting platform and Gunicorn WSGI servers.
10. **Technical Communication:** Pitch, demonstrate, and defend a digital product architecture and its community impact thesis to an industry panel on Demo Day.

Technical Tool Stack

The program relies on a modern, industry-standard technology stack. Participants will install, configure, and operate the following software, frameworks, and platforms to mirror professional software engineering environments.

Category	Tool / Platform	Application in Syllabus
Code Editors & IDEs	**Replit**	Cloud-based integrated development environment utilized for immediate, zero-setup coding and collaborative exercises.
	VS Code	Professional desktop code editor utilized for local application development, terminal access, and extension management.
	Google Colab	Interactive cloud notebooks utilized for Python data science, pandas manipulation, and data visualization.
Version Control	**Git**	Command-line version control system utilized for

		tracking local code history and reverting changes.
	GitHub	Remote repository hosting utilized for collaboration, pull requests, issue tracking, and portfolio hosting.
Design & UI/UX	**Figma / Penpot**	Collaborative interface design tools utilized for creating low-fidelity wireframes and high-fidelity clickable prototypes.
Languages & Libraries	**HTML / CSS / JS**	The core languages of the web utilized for semantic structure, styling, and client-side interactivity.
	Python 3	The primary server-side and data programming language utilized throughout the program.
	pandas / matplotlib	Python libraries utilized for data manipulation (DataFrames) and statistical charting.
Backend & Cloud	**Flask**	A lightweight Python micro-framework utilized to build REST APIs and route web traffic.
	Render	A cloud application hosting platform utilized for deploying the Flask

		backend and making the Capstone MVP live.
AI Assistants	**ChatGPT / Copilot / v0**	Generative AI platforms utilized for prompt engineering, UI generation, and coding assistance.

Weekly Schedule & Milestones

The curriculum spans seven weeks, encompassing 25 focused sessions. The structure heavily emphasizes project-based deliverables, ensuring that conceptual knowledge is immediately applied to practical output.

Master Schedule Overview

Week	Theme	Sessions	Key Deliverables & Milestones
Week 1	Coding & AI Foundations	Sessions 1–7	Working HTML/CSS/JS webpage; initial Python scripts; Capstone problem statement; GitHub repository initialized.
Week 2	Design, Planning & User Experience	Sessions 8–10	User journey map; WCAG accessibility audit; high-fidelity Figma prototype.
Week 3	Responsible AI & Data Analysis	Sessions 11–13	AI Ethics Statement; cleaned pandas dataset; data visualizations pushed to GitHub.

Week 4	Backend Development & Deployment	Sessions 14–16	Working Flask web server; .env configuration; live Render deployment URL.
Week 5	AI Integration & Collaborative Engineering	Sessions 17–19	API integration script; feature branches merged via Pull Requests; peer code reviews completed.
Week 6	Testing, Refinement & Pitch Prep	Sessions 20–22	GitHub Issues logged and resolved; AI safety guardrails implemented; initial pitch deck drafted.
Week 7	Final Build & Demo Day	Sessions 23–25	Code freeze; refined README.md; mock technical defense; final industry panel presentation.

Detailed Weekly Curriculum

Week 1: Coding & AI Foundations

The primary objective of the first week is to demystify the mechanics of the internet, introduce foundational programming syntax, and establish the habit of saving work via version control.

- **Session 1 (Welcome & Web Basics):** Explores tech career pathways, client-server architecture, and semantic HTML structure. Participants build a personal webpage and initialize their first GitHub repository with a descriptive README.md.
- **Session 2 (CSS & Visual Design):** Introduces the CSS Box Model (margin, padding, border), classes, and IDs. Participants style their webpage and practice meaningful Git commits (e.g., feat: Add homepage structure).
- **Session 3 (Responsive Layouts & AI-Assisted UI):** Focuses on Flexbox and accessibility basics. Participants prompt an AI tool (like v0 or ChatGPT) to suggest a

responsive layout, selectively integrating the code to ensure it remains accessible across mobile devices.

- **Session 4 (JavaScript Interactivity):** Transitions the DOM from static to dynamic. Variables, functions, and event listeners (getElementById) are utilized to build a simple interactive element, such as a calculator or quiz.
- **Session 5 (Introduction to Python):** Shifts to backend logic utilizing Google Colab. Participants cover variables, strings, floats, booleans, and conditional statements. An AI mini-task is introduced to summarize text, requiring human verification.
- **Session 6 (Python Data Structures):** Expands into lists, indexing, dictionaries, and loops. Participants build a digital contact book and compare rule-based classification scripts against AI-prompted classification.
- **Session 7 (Capstone Ideation):** Teams are formed to identify a Maldivian or community problem. The Minimum Viable Product (MVP) scope is finalized, distinguishing between "core" features and "nice-to-have" additions.

Week 2: Design, Planning & User Experience

Before writing backend code, the product must be architected for human usability. This week focuses entirely on user journeys, wireframing, and inclusive design standards.

- **Session 8 (UI/UX & User Journey):** Distinguishes between User Interface (UI) and User Experience (UX). Participants map out the user journey and sketch low-fidelity wireframes.
- **Session 9 (Accessibility & Inclusive Design):** A deep dive into the Web Content Accessibility Guidelines (WCAG) and the POUR principles:
  - **Perceivable:** Information must be presented so users can perceive it (e.g., alt text for screen readers, 4.5:1 color contrast ratios).
  - **Operable:** Interfaces cannot require interactions a user cannot perform. Participants ensure keyboard navigability and verify that mobile touch targets meet the WCAG 2.2 AA requirement of at least 24×24 CSS pixels to prevent accidental taps.
  - **Understandable:** Information and operation must be predictable (e.g., clear form labels and visible error identification).
  - **Robust:** Content must remain compatible with current and future assistive technologies.
- **Session 10 (Figma Prototype):** Wireframes are converted into high-fidelity, clickable prototypes. Reusable components are established, and participants begin coding the frontend of their Capstone.

Week 3: Responsible AI & Data Analysis

Data is the foundation of both smart civic planning and artificial intelligence. This week equips participants to process raw data responsibly.

- **Session 11 (Responsible AI, Privacy & Ethics):** Explores UNESCO AI competency frameworks, bias, hallucinations, and data protection. Participants analyze scenarios where AI classifications fail and design workflows that enforce human-in-the-loop approvals before sensitive actions occur.
- **Session 12 (Data Analysis with Python):** Introduces the pandas library for working with CSV and JSON files. Participants load DataFrames, handle missing values, and extract medians and counts to derive insights.
- **Session 13 (Data Visualization):** Utilizes matplotlib and seaborn to generate scatter

plots, bar charts, and distributions. Participants automate a draft narrative summary of the data using an AI API, manually verifying the output against the charts.

Week 4: Backend Development, Git & Deployment

The frontend designs and Python logic are unified into a functional web application and pushed to the live internet.

- **Session 14 (Flask Framework):** Introduces the Flask micro-framework. Participants learn routing, HTTP methods (GET vs. POST), and Jinja templating. Forms are connected to backend Python functions to process user input.
- **Session 15 (Git & GitHub Full Workflow):** Moves beyond the browser into terminal-based version control. Participants master the local-to-remote pipeline (git init, add, commit, remote add origin, push). The concept of keeping a linear, clean commit history is emphasized.
- **Session 16 (Error Handling & Deployment):** Focuses on application robustness. Participants implement try/except blocks to handle Python tracebacks gracefully. Environment variables (.env) are configured to hide secrets, and the application is deployed to Render using Gunicorn as the WSGI HTTP server.

Week 5: AI Integration & Collaborative Engineering

Individual coding transitions into team-based software engineering, mimicking professional development environments.

- **Session 17 (APIs & AI Integration):** Bridges the gap from "using AI" to "automating with AI". Participants connect their Flask app to external REST APIs, parsing JSON responses to perform text summarization or classification.
- **Session 18 (Prompt Engineering & Vibe Coding):** Teaches advanced prompt structuring. Participants design an automation flow diagram outlining trigger → processing → AI call → structured output → human review.
- **Session 19 (GitHub Collaboration):** Introduces branching (git checkout -b) and Pull Requests (PRs). Participants learn to review peer code constructively. Best practices require keeping PRs focused (ideally under 50 lines), providing context in the PR description, and utilizing review templates.

Week 6: Testing, Refinement & Pitch Preparation

The final development sprint focuses on stabilizing the MVP, managing defects, and translating technical architecture into a compelling narrative.

- **Session 20 (Testing & GitHub Issues):** Introduces Quality Assurance (QA). Participants conduct cross-browser and responsive testing, logging bugs as GitHub Issues and assigning them to team members for resolution.
- **Session 21 (AI Safety & Secrets Management):** Focuses on security. The application is stress-tested against prompt injections and unexpected inputs. Participants audit their .gitignore files to guarantee no API keys or database credentials are exposed to public repositories.
- **Session 22 (Pitch & Demo Preparation):** Teams construct a pitch deck outlining the Hook → Problem → Solution → Demo → Architecture → Responsible AI framework. A timed, live end-to-end demonstration is rehearsed to identify friction points.

Week 7: Final Build & Demo Day

The program concludes by preparing the application for public presentation and defending the

technical decisions.

- **Session 23 (Final Code Review):** A strict code freeze is implemented. The repository is cleaned, dependencies are locked in a requirements.txt file, and the README.md is polished to serve as a professional portfolio asset.
- **Session 24 (Mock Demo & Technical Defense):** Teams execute full mock presentations. Facilitators conduct technical Q&A sessions, ensuring participants can articulate their architecture, explain how their AI integration functions, and defend their ethical safeguards.
- **Session 25 (Demo Day):** The culmination of the program. Teams present the live MVP, GitHub portfolio, and pitch deck to a panel of industry experts, demonstrating their journey from zero coding experience to deployed digital product creators.

The Capstone Project: Minimum Viable Product (MVP)

The Capstone Project is the core pedagogical vehicle of the Girls to Code program. Rather than completing isolated exercises, participants dedicate the entire seven weeks to building a single, deployable digital product.

Core MVP Requirements

1. Community Problem Identification:

The software must address a tangible issue. Drawing inspiration from local contexts—such as Malé's dense urban environment, climate resilience needs, digital literacy gaps, or small business challenges—teams must articulate a clear problem statement and target demographic.

2. Strict Feature Focus:

A common pitfall in software development is scope creep. The Capstone strictly limits the MVP to 3 to 5 core features. The objective is a stable, bug-free, and highly polished micro-application rather than a sprawling, unfinished project.

3. The Automated AI/Logic Step:

The product must integrate an automated workflow that reduces manual effort. Participants are required to map out and implement a clear logic pipeline:

- **Input/Trigger:** User submits a form, uploads a document, or triggers an event.
- **Processing:** Flask backend receives the data and sanitizes the input.
- **AI/API Step:** The server securely queries an external API (e.g., OpenAI) utilizing environment variables.
- **Structured Output:** The AI returns structured data (e.g., JSON).
- **Action & Human Review:** The application displays the result, requiring human validation before the data is permanently stored or acted upon.

4. Portfolio-Ready Repository:

The final submission is not just the live website, but the GitHub repository itself. The codebase must feature a comprehensive README.md, a clean commit history, resolved GitHub Issues, and evidence of peer code reviews.

Everyday Habits: The GitHub & AI Tracks

Technical proficiency is built through daily repetition. The syllabus mandates the continuous use of Version Control and AI integration, transforming them from abstract concepts into everyday habits.

The Progressive GitHub Workflow

Participants will transition from individual file saves to professional, team-based version control over the seven weeks.

1. Resolving the "Everyday" Cycle (Weeks 1-3):

Participants are conditioned to run the standard save cycle multiple times per session:

- git status (Verify modified files)
- git add . (Stage all changes)
- git commit -m "Descriptive action" (Commit to local history)
- git push (Upload to remote repository)

2. Collaborative Branching & Pull Requests (Weeks 4-7):

When teams begin working on the same Capstone codebase, direct pushes to the main branch are restricted. Participants must utilize feature branches:

- git checkout -b feature/login-page (Create an isolated branch)
- *Write and test code locally...*
- git push -u origin feature/login-page (Push the branch to GitHub)

Once pushed, the participant opens a Pull Request (PR). The syllabus enforces professional PR best practices:

- **Small Scope:** PRs should ideally remain under 50 lines of code to prevent reviewer fatigue.
- **Clear Context:** PR descriptions must explain *why* the change was made, detailing the database, API, or frontend adjustments.
- **Constructive Review:** Reviewers are taught to critique the code, not the coder, utilizing tools to automate formatting checks and focusing human review on logic, architecture, and security.

3. Handling Merge Conflicts:

As multiple developers edit the same files, merge conflicts are inevitable. Participants are trained to read the conflict markers (<<<<<<< HEAD), communicate with their peers to determine which code to keep, manually resolve the conflict, and complete the merge commit rather than panicking or deleting the repository.

Debugging & Python Error Handling

Beginners often find error messages intimidating. The curriculum teaches participants to treat errors as diagnostic tools rather than failures.

- **Reading Tracebacks:** Participants learn to read Python tracebacks from the bottom up. The traceback indicates the exact file, line number, and type of error (e.g., SyntaxError, KeyError), showing the sequence of function calls that led to the crash.
- **Graceful Degradation:** Instead of allowing the Flask server to crash when an API fails or bad data is submitted, participants implement try/except blocks to catch exceptions, logging the error internally while returning a user-friendly JSON message or HTML error page to the frontend.

The Progressive AI Automation Journey

AI is integrated not as a shortcut to bypass learning, but as a force multiplier for productivity and complex problem-solving.

- **Stage 1: AI as a Helper (Weeks 1-3):** Participants utilize AI tools to brainstorm UI layouts, explain confusing syntax, or generate boilerplate code. The absolute rule is that participants must be able to explain any code they commit.

- **Stage 2: Simple Automation (Weeks 4-5):** Participants write Python scripts that interface with AI to perform bulk tasks, such as summarizing a dataset of user feedback or extracting specific fields from unstructured text.
- **Stage 3: Safe, Integrated Automation (Weeks 6-7):** The Capstone MVP connects to AI APIs. Participants must define structured JSON schemas to ensure the AI returns data the application can predictably parse. Crucially, participants implement safety guardrails, sanitizing user inputs to prevent prompt injections and ensuring that the AI's output is flagged for human review if confidence thresholds are not met.

Through this rigorous, project-driven methodology, the Girls to Code 2026 syllabus ensures that participants do not just learn to write code; they learn to engineer robust, ethical, and community-focused digital products.